

Column-Fit Test: Inconsolata variant

Tim Menzies

1 CONFIG AND CSV

```
#!/usr/bin/env python3 -B
#HashBang
# pyright: reportAttributeAccessIssue=false
# pyright: reportOperatorIssue=false, reportArgumentType=false
5 #StaticBugs
"""
flair: explainable multi-objective active learning
(c) 2026 Tim Menzies <timme@ieee.org> MIT license

10 Options:

    --p=2            minkowski coefficient
    --few=256        max rows scanned inside one tree leaf
    --stop=20        stopping rule for recursive tree generation
15  --lots=2048       max rows used to build any tree
    --budget=50      total labelling budget for optimizing
    --check=30       optimization: how many top picks to label
    --est=mean       holdout: leaf estimate; near(est) or mean
    --more=4         acquisition: labels per round
20  --best=.66       acquisition: pool keep fraction
    --seed=1234      random number generation
    --places=2       show these decimal places
    --file=/Users/timm/gits/moot/optimize/misc/auto93.csv
"""
25 #SSOT #Config #Regx #DSL
import re, os, sys, glob, traceback
from math import exp, log
from random import seed, sample
from types import SimpleNamespace as Box
30 import sys; sys.dont_write_bytecode = True
#Nocache

BIG = 1e30

35 def atom(s): # string --> number or stripped string
    for f in [int, float]:
        try: return f(s)
        except ValueError: ...
    return s.strip()

40 the = Box(**{k: atom(v) for k, v in
               re.findall(r"(\w+)=(\S+)", __doc__ or "")})

def csv(file): # iterate a csv file's atom rows
45     with open(file) as f:
        for s in f:
            if s := s.strip():
                yield [atom(x) for x in s.split(",")]
```

2 TYPES AND UPDATE

```
# --- create: all types are a Box, tagged by their maker -----
50 def Num(): return Box(it=Num, n=0, mu=0, m2=0, sd=0)
def Sym(): return Box(it=Sym, n=0, has={})
def Col(s): return (Num if s[0].isupper() else Sym)()
def Tbl(): return Box(it=Tbl, rows=[], cols=None)

55 def Cols(names): # names --> columns grouped into x,y
    i = Box(it=Cols, names=names, all=[], x={}, y={}, klass=None)
    for at, s in enumerate(names):
        i.all += [Col(s)]
```

```
    if s[-1] == "X": pass
60  elif s[-1] == "!": i.klass = at
    elif s[-1] in "+-": i.y[at] = s[-1] == "+"
    else: i.x[at] = at
    return i

65 def clone(tbl, rows=[]): # new table, same structure as tbl
    return adds([tbl.cols.names] + rows, Tbl())

# --- add: update -----
def adds(src, it=None): # add all from any iterable
70  it = it or Num()
    for x in src: add(it, x)
    return it

def add(i, v, w=1): # update any box; w=-1 is deletion
75  if i.it is Tbl:
        if i.cols: add(i.cols, v, w); i.rows += [v]
        else: i.cols = Cols(v)
    elif i.it is Cols:
        [add(c, x, w) for c, x in zip(i.all, v) if x != "?"]
80  else:
        i.n += w
        if i.it is Sym: i.has[v] = w + i.has.get(v, 0)
        elif i.n < 1: i.n = i.mu = i.m2 = i.sd = 0
        else:
            d = v - i.mu
            i.mu += w*d/i.n
            i.m2 = max(0, i.m2 + w*d*(v - i.mu))
            i.sd = 0 if i.n < 2 else (i.m2/(i.n - 1))**.5

90 # --- mid, div: central tendency, diversity -----
def mid(col): # Num: mean. Sym: mode
    return (col.mu if col.it is Num else
            max(col.has, key=col.has.get))

95 def mids(tbl): return [mid(col) for col in tbl.cols.all]

def div(col): # Num: sd. Sym: entropy (v>0 dodges deletions)
    return col.sd if col.it is Num else \
        -sum(v/col.n*log(v/col.n, 2) for v in col.has.values() if v>0)

100 def norm(num, v): # Num value --> 0..1, logistic cdf
    if v == "?": return v
    z = (v - num.mu)/(num.sd + 1/BIG)
    return 1/(1 + exp(-1.7*max(-3, min(3, z))))
```

3 DISTANCE AND DESCENT

```
105 # --- dist: distance -----
def _dist(col, a, b): # one column's distance
    if a == "?" and b == "?": return 1
    if col.it is Sym: return a != b
    a, b = norm(col, a), norm(col, b)
110  if a == "?": a = 1 if b < 0.5 else 0
    if b == "?": b = 1 if a < 0.5 else 0
    return abs(a - b)

def distx(tbl, r1, r2): # x-column distance
115  d, n = 0, 1/BIG
    for at in tbl.cols.x:
        n += 1
        d += _dist(tbl.cols.all[at], r1[at], r2[at])**the.p
```

```

return (d/n)**(1/the.p)

120 def disty(tbl, row): # d2h: distance of goals to best corner
    d = sum(abs(norm(tbl.cols.all[at], row[at]) - w)**the.p
            for at, w in tbl.cols.y.items())
    return (d/len(tbl.cols.y))**(1/the.p)

125 def ymids(tbl, rows): # mids of the y columns, in these rows
    return [mid(adds((r[at] for r in rows if r[at] != "?"),
                    tbl.cols.all[at].it())) for at in tbl.cols.y]

130 # --- descend: label a few rows, cull toward the good pole -----
def poles(tbl, rows, y): # far pair in rows; best-->worst axis
    far = lambda r: max(rows, key=lambda x: distx(tbl, x, r))
    a = far(rows[0]); z = far(a)
    if y(z) < y(a): a, z = z, a
135 c = distx(tbl, a, z) + 1/BIG
    return lambda r: (distx(tbl, a, r)**2 + c*c -
                    distx(tbl, z, r)**2)/(2*c)

def descends(tbl, rows, label=lambda row: row):
140 def descend(rows): # one greedy descent along project
    while len(rows) > the.stop and len(lab) < cap:
        todo, more = [], min(the.more, cap - len(lab))
        for r in rows:
            if id(r) in lab: todo += [lab[id(r)]]
145         elif more > 0:
            more -= 1; lab[id(r)] = label(r); todo += [lab[id(r)]]
        rows = sorted(rows, key=project(todo))
        rows = rows[:int(the.best*len(rows))]

150 y = lambda r: disty(tbl, r)
    project = lambda rows: poles(tbl, rows, y)
    lab = {}
    cap = the.budget - the.check
    while len(lab) < cap:
        n = len(lab)
        descend(shuffle(rows))
        if len(lab) == n: break # no progress
    return sorted(lab.values(), key=y)

```

4 CUTS AND TREES

```

# --- cut: min expected variance splits -----
160 def matches(col, x, v): # does x fall on the yes side of cut v?
    return x == "?" or (x == v if col.it is Sym else x <= v)

def selects(z, row): return matches(z.col, row[z.at], z.v)

165 def score(col1, col2): # expected diversity of col1|col2 split
    n1, n2 = col1.n, col2.n
    return (div(col1)*n1 + div(col2)*n2)/(n1 + n2 + 1/BIG)

def cutsSym(xy, tot, acc): # (score,v): yes = one symbol
170 for v in dict.fromkeys(x for x, _ in xy):
    lhs = adds((y for x, y in xy if x == v), acc())
    rhs = adds((y for x, y in xy if x != v), acc())
    yield score(lhs, rhs), v

175 def cutsNum(xy, tot, acc): # (score,v) per bound, one sweep
    xy.sort()
    lhs = acc()
    for j, (x, y) in enumerate(xy):
        add(lhs, y); add(tot, y, -1) # rhs = shrinking tot
        if j + 1 < len(xy) and x != xy[j+1][0]:
180         yield score(lhs, tot), x

def cutsTbl(tbl, rows, y, acc=Num): # (score,at,v), all x cols
    for at in tbl.cols.x:
185     xy = [(r[at], y(r)) for r in rows if r[at] != "?"]
        tot = adds((y for _, y in xy), acc())
        f = cutsNum if tbl.cols.all[at].it is Num else cutsSym
        for s, v in f(xy, tot, acc):
            yield s, at, v

190 def cutTbl(tbl, rows, y, acc=Num): # best cut, as a labeled Span
    if z := min(cutsTbl(tbl, rows, y, acc), default=None):
        _, at, v = z
        c, s = tbl.cols.all[at], tbl.cols.names[at]
        eq, ne = ("==", "!=") if c.it is Sym else ("<=", ">")
195         return Box(it=cutTbl, at=at, v=v, col=c,
                    txt=f"{s} {eq} {v}", anti=f"{s} {ne} {v}")

```

```

# --- tree -----
200 def Tree(**d):
    return Box(**dict(it=Tree, n=0, rows=[], cut=None,
                    ys=None, leafs=1) | d)

def growTree(tbl, y=None, acc=Num): # min-variance splits
    y = y or (lambda r: disty(tbl, r))
    def recurse(rows):
        node = Tree(n=len(rows), rows=rows, ys=adds(map(y, rows), acc()))
        if len(rows) > the.stop:
            if z := cutTbl(tbl, rows, y, acc):
                yes, no = [], []
                for r in rows: (yes if selects(z, r) else no).append(r)
                if yes and no:
                    node.cut, node.yes, node.no = z, recurse(yes), recurse(no)
                    node.leafs = node.yes.leafs + node.no.leafs
215         return node
    return recurse(tbl.rows)

def leaf(tree, row): # walk row down to its leaf
    while tree.cut:
220     tree = tree.yes if selects(tree.cut, row) else tree.no
    return tree

def predict(tree, row): # leaf's mode (Sym) or mean (Num)
    return mid(leaf(tree, row).ys)

225 def guess(tree, tbl, row): # d2h of leaf row nearest to mids
    l = leaf(tree, row)
    if not hasattr(l, "est"):
        c = mids(clone(tbl, l.rows))
        l.est = disty(tbl, min(some(l.rows, the.few),
230         key=lambda r: distx(tbl, r, c)))
    return l.est

def nodes(tree, pre=None, txt=""): # walk: (node, indented txt)
235 yield tree, (pre or "") + txt
    if tree.cut:
        sub = "" if pre is None else pre + "| "
        yield from nodes(tree.yes, sub, tree.cut.txt)
        yield from nodes(tree.no, sub, tree.cut.anti)

240 def showTree(tree, tbl): # y-col mids per node; +/- bad,best leaf
    ns = list(nodes(tree))
    leafs = [n for n, _ in ns if not n.cut]
    best = min(leafs, key=lambda n: mid(n.ys))
    worst = max(leafs, key=lambda n: mid(n.ys))
    mark = lambda n: ("+" if n is best else
245     "-" if n is worst else "")
    printm(["", "d2h", "n",
            *[tbl.cols.names[at] for at in tbl.cols.y], "" ] +
            [mark(n), mid(n.ys), n.n, *ymids(tbl, n.rows), txt]
            for n, txt in ns],
            "<>" + ">"*len(tbl.cols.y))

```

5 MISC AND STATS

```

# --- misc -----
def shuffle(t): return sample(t, len(t)) # non-mutating
255 def some(t, n): # n random picks from list t, no repeats
    return sample(t, min(n, len(t)))

def o(v): # tidy: round floats; boxes --> dicts, no _keys
    if type(v) is float:
        return int(v) if v == int(v) else round(v, the.places)
    if callable(v): return v.__name__
    if hasattr(v, "__dict__"):
        return {k: o(w) for k, w in vars(v).items() if k[0] != "_"}
260     return v

def oo(x): print(o(x)); return x

270 def printm(rows, align=""): # align columns; flags "<->" per col
    rows = [[str(o(x)) for x in r] for r in rows]
    ws = [max(map(len, c)) for c in zip(*rows)]
    aligns = align.replace("-", "").ljust(len(ws), "<")
    for r in rows:
        print(" ".join(f"{x:{a}{w}}"
275         for x, a, w in zip(r, aligns, ws)).rstrip())

```

```

def wins(t, rows=None): # grader: row --> % of gap closed
    ys = sorted(disty(t, r) for r in rows or t.rows)
    lo, b4 = ys[0], sum(ys)/len(ys)
    return lambda r: max(-100, min(100,
    100*(1 - (disty(t, r) - lo)/(b4 - lo + 1/BIG))))

def holdout(t): # train: half, capped at lots. test: the rest
    rows = shuffle(t.rows)
    n = len(rows)//2
    tr = clone(t, descends(t, rows[:n]:the.lots]))
    tt = growTree(tr)
    est = ((lambda r: predict(tt, r)) if the.est == "mean"
    else lambda r: guess(tt, tr, r))
    top = sorted(rows[n:], key=est)
    return min(top[:the.check],
    key=lambda r: disty(tr, r)), rows[n:], tr

# --- stats: are two samples of numbers the same? -----
def cohen(xs, ys, d=0.35): # mean gap small, in pooled sd units
    x, y = adds(xs), adds(ys)
    sd = (((x.n-1)*x.sd**2 + (y.n-1)*y.sd**2)/(x.n+y.n-2))**0.5
    return abs(x.mu - y.mu) <= d*sd

def cliffs(xs, ys, d=0.197): # sorted xs,ys: rank imbalance.
    gt = lt = j = k = 0 # j,k = ys below, at-or-below x;
    for x in xs: # they only ever advance
        while j < len(ys) and ys[j] < x: j += 1; k = j
        while k < len(ys) and ys[k] <= x: k += 1
    gt += j; lt += len(ys) - k
    return abs(gt - lt)/(len(xs)*len(ys)) <= d

def ks(xs, ys, a=1.36): # sorted xs,ys: 95% kolmogorov-smirnov
    n, m, i, j, d = len(xs), len(ys), 0, 0, 0
    while i < n and j < m:
        v = min(xs[i], ys[j])
        while i < n and xs[i] <= v: i += 1
        while j < m and ys[j] <= v: j += 1
    d = max(d, abs(i/n - j/m))
    return d <= a*((n + m)/(n*m))**0.5

def same(xs, ys): # indistinguishable by all three tests
    xs, ys = sorted(xs), sorted(ys)
    return cliffs(xs, ys) and ks(xs, ys) and cohen(xs, ys)

def top(d, reverse=False): # names statistically tied with best
    mu = lambda a: sum(a)/len(a)
    xs = sorted(d.items(), key=lambda kv: mu(kv[1]), reverse=reverse)
    j = 0
    while j+1 < len(xs) and same(xs[j][1], xs[j+1][1]): j += 1
    return {k for k, _ in xs[:j+1]}

```

6 DEMOS AND MAIN

```

# --- demos -----
def test_list():
    "show the demos"
    for k, f in eg.items():
        print("%-12s %s" % (k, (f.__doc__ or "").strip()))

def test_all():
    "run all the demos; exit 1 if any crash"
    bad = 0
    for k, f in eg.items():
        if f not in (test_all, test_push):
            print("\n#", k); bad += run(f)
    print("\n%s failure(s)" % bad)
    sys.exit(bad > 0)

def test_push():
    "git commit -am saving; git push; git status"
    os.system("git commit -am saving; git push; git status")

def test_the():
    "show current settings"
    oo(the)

def test_atom():
    "strings coerce to numbers or stripped strings"
    assert atom("2") == 2 and atom("2.1") == 2.1
    assert atom(" a ") == "a"
    print("'2' ->", atom("2"), " | ' a ' ->", atom(" a "))

```

```

def test_csv():
    "csv reader finds many rows in the file"
    n = sum(1 for _ in csv(the.file))
    assert n > 100
    print(n, "rows")

def test_cuts():
    "best (least variance) cut of full table"
    t = adds(csv(the.file), Tbl())
    s, at, v = min(cutsTbl(t, t.rows, lambda r: disty(t, r)))
    print(t.cols.names[at], "at", v, "score %.3f" % s)

def errs(t, tr, tt, rows): # prediction errors, one holdout
    return ((abs(guess(tt, tr, r) - disty(tr, r))
    for r in rows))

def test_predict():
    "20 runs: mu, sd of |predicted - actual| d2h"
    t, err = adds(csv(the.file), Tbl()), Num()
    for _ in range(20):
        rows = shuffle(t.rows)
        n = len(rows)*2//3
        tr = clone(t, rows[:n])
        adds(errs(t, tr, growTree(tr), rows[n:], err))
    print("err mu %.3f sd %.3f" % (err.mu, err.sd))

def test_err():
    "20 moot sets, 20 repeats: err mu/sd, mean leaf count"
    fs = sorted(glob.glob(
        "/Users/timm/gits/moot/optimize/**/*.*.csv",
        recursive=True), key=os.path.getsize)[:20]
    for f in fs:
        t = adds(csv(f), Tbl())
        err, nleaf = Num(), Num()
        for _ in range(20):
            rows = shuffle(t.rows)
            n = len(rows)*2//3
            tr = clone(t, rows[:n])
            tt = growTree(tr)
            add(nleaf, tt.leaves)
            adds(errs(t, tr, tt, rows[n:], err))
        print("%-22s %5s %4.0f err %2.0f (%2.0f)" %
            (f.split("/")[-1][:22], len(t.rows), nleaf.mu,
            100*err.mu, 100*err.sd))

def opt1(f): # one dataset: win of tree, rand, best picks
    t = adds(csv(f), Tbl())
    w = wins(t)
    ts, rs, bs = [], [], []
    for _ in range(20):
        got, test, tr = holdout(t)
        y = lambda r: disty(tr, r)
        ts += [w(got)]
        rs += [w(min(some(test, the.budget), key=y))]
        bs += [w(min(test, key=y))]
    treat, rand, best = adds(ts), adds(rs), adds(bs)
    d = 0 if same(ts, rs) else treat.mu - rand.mu
    print("%-22s %5s tree %4.0f rand %4.0f best %4.0f diff %4.0f" %
        (f.split("/")[-1][:22], len(t.rows),
        treat.mu, rand.mu, best.mu, d))

def test_opt():
    "optimize the 20 smallest moot data sets"
    fs = sorted(glob.glob(
        "/Users/timm/gits/moot/optimize/**/*.*.csv",
        recursive=True), key=os.path.getsize)[:20]
    stop0, the.stop = the.stop, 10
    for f in fs: opt1(f)
    the.stop = stop0

def test_opt1():
    "optimize just the file (for parallel sweeps)"
    stop0, the.stop = the.stop, 10
    opt1(the.file)
    the.stop = stop0

def test_tree():
    "recursive contrast splits; leaves show row counts"
    t = adds(csv(the.file), Tbl())
    showTree(growTree(t), t)

def test_klass():
    "classify diabetes: accuracy of a klass tree, 5 holdouts"
    t = adds(csv("/Users/timm/gits/moot/classify/diabetes.csv"),
    Tbl())

```

```

y = lambda r: r[t.cols.klass]
acc = Num()
for _ in range(5):
    rows = shuffle(t.rows)
    n = len(rows)*2//3
    tt = growTree(clone(t, rows[:n]), y=y, acc=Sym)
    add(acc, sum(predict(tt, r) == y(r)
                  for r in rows[n:])/len(rows) - n))
450 print("accuracy mu %.2f sd %.2f" % (acc.mu, acc.sd))

# --- main -----
eg = {"_" + k[5:]: f for k, f in globals().items()
      if k.startswith("test_")}

455 def run(f): # reseed, call f, catch crashes; 1 if crashed
    seed(the.seed)
    try: f()
    except Exception: traceback.print_exc(); return 1
460 return 0

def runs():
    errs = 0
    for j, s in enumerate(sys.argv):
        if s=="-h": print(__doc__)
        if f := eg.get("-" + s.lstrip("-")):
            errs += run(f)
        elif hasattr(the, k := s.lstrip("-")):
            setattr(the, k, atom(sys.argv[j + 1]))
470 return errs

if __name__ == "__main__": sys.exit(runs())

```